دليل تشغيل كود الـ RAG

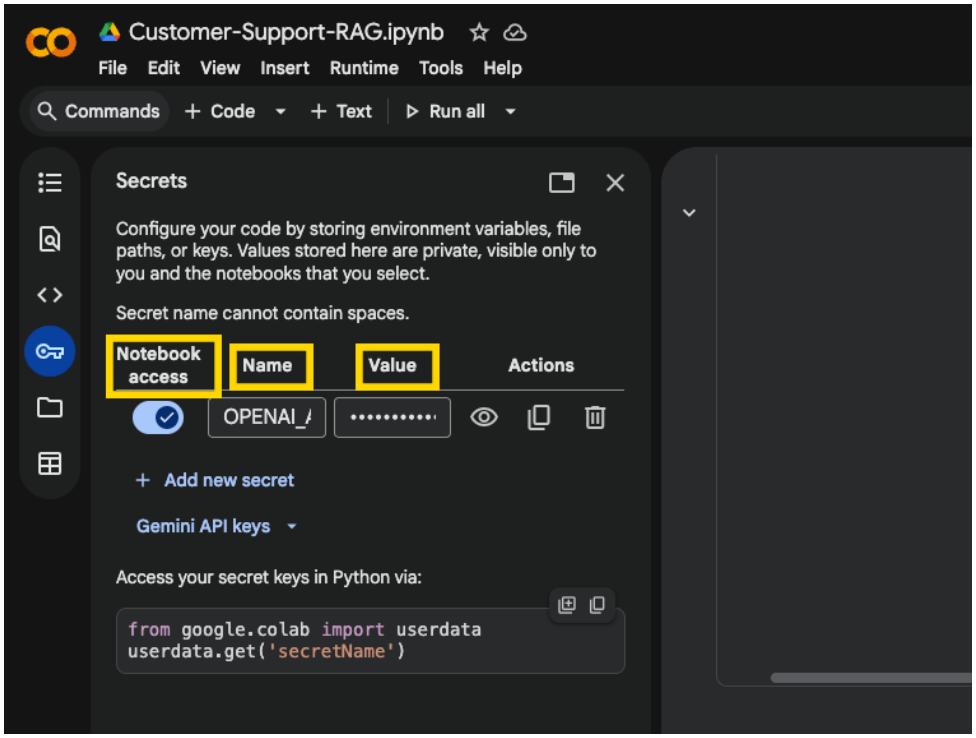
راجع الفيديو من الرابط التالي لسهولة التنفيذ: اضغط هنا
الكود الكامل: اضغط هنا

الخطوة رقم ١ والأهم

احضار مفتاح الـ API من موقع OpenAI

١. لاحضار المفتاح راجع الفيديو من الدقيقة 14:47
٢. ارجع علي جوجل كولاب
٣. علي اليسار اضغط علي الأيقونة اللي عليها رمز المفتاح
٤. اضغط على Add new secret
٥. في خانة Name اكتب: `OPENAI_API_KEY`
٦. وفي خانة Value الصق المفتاح اللي نسخته
٧. فعل خيار Notebook access حتى يتحول للون الأزرق ويظهر عليه علامة صح

صورة توضيحية:



الخطوة رقم ٢

تثبيت المكتبات المهمة والضرورية للتشغيل

شغل الخلية الخاصة بتثبيت الـ Libraries زي:

- LangChain
- Chroma
- PyPDF
- Gradio
- OpenAI SDK

صورة توضيحية:

```
!pip install -q --upgrade langchain langchain-openai langchain-community langchain-chroma \
  langchain-huggingface langchain-text-splitters chromadb python-dotenv sentence-transformers gradio pypdf

!pip install -q requests==2.32.4

!pip install -q opentelemetry-exporter-otlp-proto-grpc posthog pybase64
```

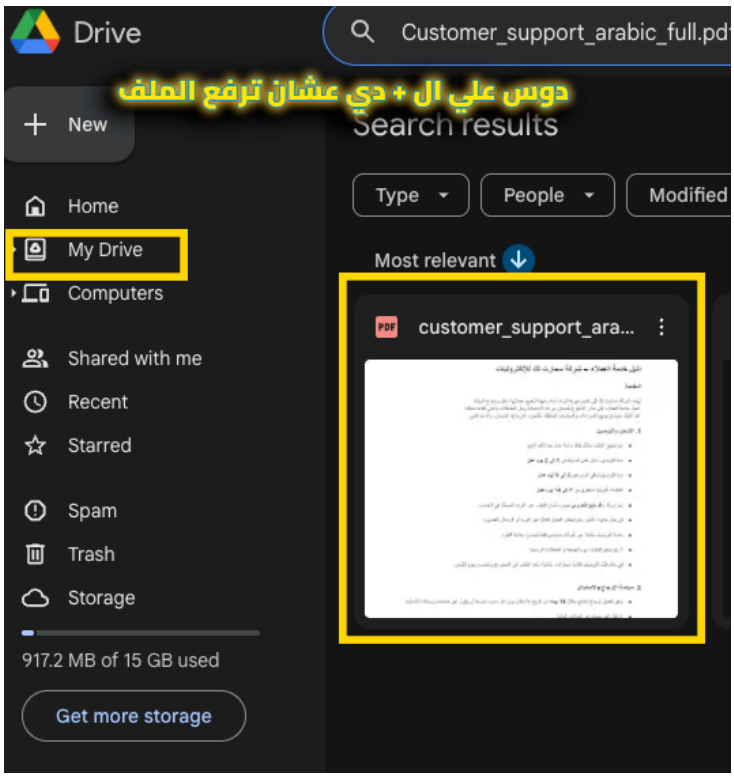
الخطوة رقم ٣

رفع ملف البيانات وتوصيل جوجل درايف

١. ارفع ملف الـ PDF الخاص ببياناتك على Google Drive

٢. يفضل وضعه في MyDrive مباشرة

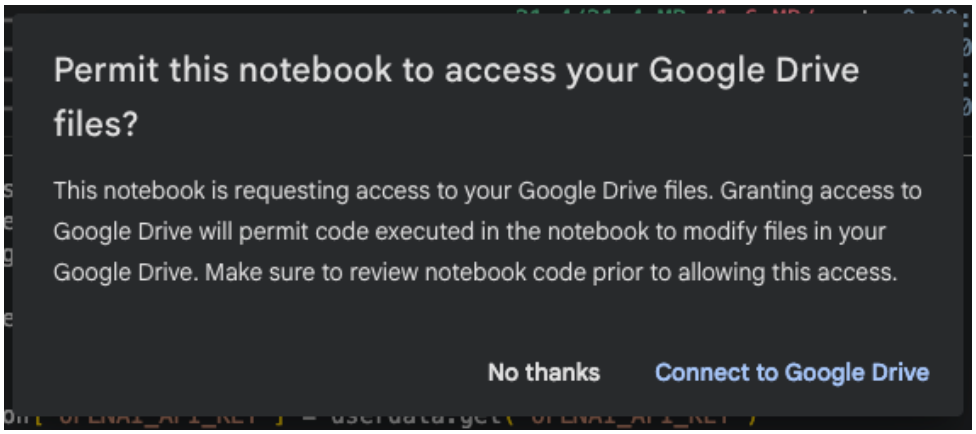
صورة توضيحية:

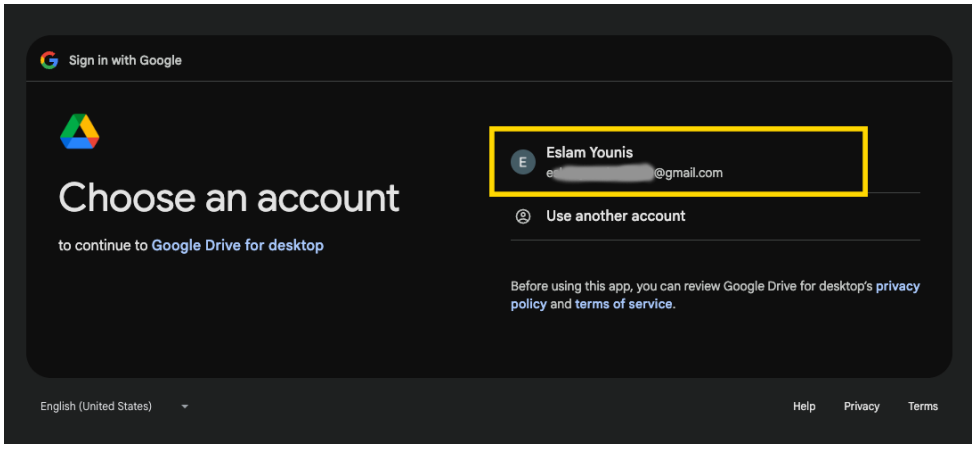


٣. بعد ذلك وصل الـ Notebook بالـ Drive باستخدام الكود التالي:

```
from google.colab import drive
drive.mount('/content/drive')
```

بعد التشغيل سيطلب منك اختيار حساب جوجل درايف الموجود عليه الملف، برجاء اختيار الحساب الصحيح.
صور توضيحية:





الخطوة رقم ٤

تعديل مسار ملف البيانات والتأكد من وجوده

افتح خلية الكود اللي فيها تعريف:

- PDF_PATH
- CHROMA_DIR

وتأكد من:

- مسار ملف الـ PDF صحيح
- الملف موجود بالفعل في المكان اللي كتبته

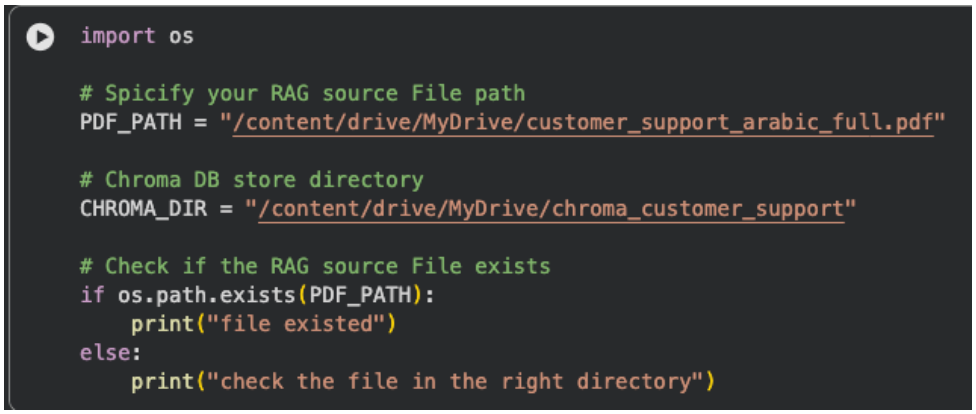
مثال:

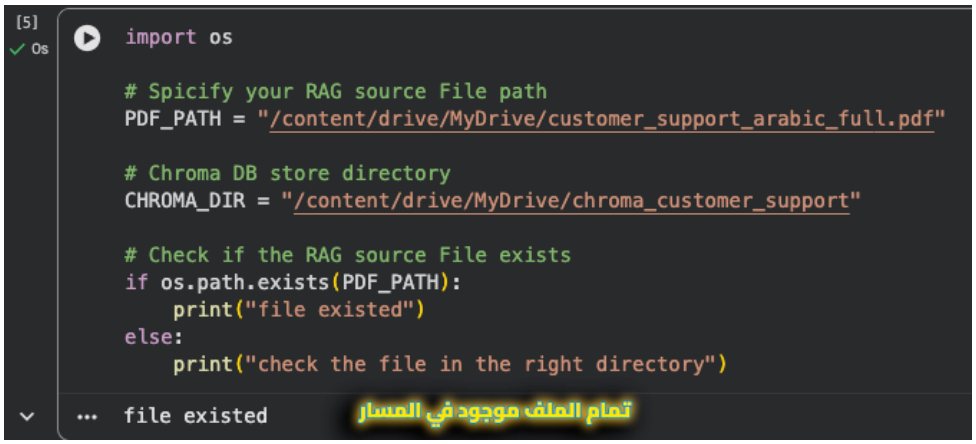
```
PDF_PATH = "/content/drive/MyDrive/customer_support_arabic_full.pdf"
CHROMA_DIR = "/content/drive/MyDrive/RAG_DB"
```

بعد ذلك شغل الخلية:

- لو ظهرت file existed يبقى تمام
- لو ظهرت check the file in the right directory راجع المسار مرة كمان

صور توضيحية:





الخطوة رقم ٥

تحميل ملف PDF وتقسيمه وبناء قاعدة بيانات Chroma

في الخلايا التالية الكود ينفذ الآتي:

- تحميل الملف باستخدام PyPDFLoader
- تقسيم النص إلى أجزاء صغيرة chunks

صورة توضيحية:

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings

loader = PyPDFLoader(PDF_PATH)
docs = loader.load()

splitter = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=150)
chunks = splitter.split_documents(docs)
```

- إنشاء Embeddings باستخدام

تُعد نماذج التضمين (Embedding Models) بمثابة "المترجم الذكي" للآلة. بما أن الحاسوب لا يعالج اللغة كنصوص مجردة، فإن هذه النماذج تقوم بتحويل الجمل إلى متجهات رقمية (Vectors) تعبر عن **المعنى الدلالي** للنص، مما يسمح للنظام بالربط بين الكلمات المترادفة (مثل: "سعر" و"تكلفة") حتى لو اختلفت الألفاظ المستخدمة في البحث عن الموجودة في المستندات.

أهمية اختيار النموذج المناسب للغة العربية: من الضروري اختيار نموذج يدعم اللغة المستخدمة في البيانات بدقة. استخدام نموذج مخصص للغة الإنجليزية فقط مع نصوص عربية سيؤدي إلى ضعف دقة النتائج. إليك مقارنة لأهم النماذج المتاحة في الكود:

- **sentence-transformers/all-MiniLM-L6-v2**: نموذج شهير وسريع جداً، ولكنه مدرب بشكل أساسي على اللغة الإنجليزية. أداؤه مع اللغة العربية محدود وقد لا يلتقط المعاني العميقة، لذا يُنصح به فقط إذا كانت الملفات المستخدمة باللغة الإنجليزية.
- **intfloat/multilingual-e5-small** (الخيار المجاني المفضل): نموذج متعدد اللغات (Multilingual)، تم تدريبه ليفهم اللغة العربية بكفاءة عالية. يُعد الخيار الأفضل إذا كنت تبحث عن نموذج مجاني ومفتوح المصدر يعمل محلياً.
- **text-embedding-3-small** (الخيار الاحترافي): أحدث نماذج شركة OpenAI. يتميز بدقة فائقة في فهم اللغة العربية (بما في ذلك المصطلحات العامية). هو الخيار الأمثل للمشاريع الإنتاجية التي تتطلب أعلى درجات الدقة (يتطلب مفتاح API ورصيداً بسيطاً).

صورة توضيحية:

```
from langchain_chroma import Chroma

embeddings = HuggingFaceEmbeddings(model_name="intfloat/multilingual-e5-small")
```

- بناء أو تحميل قاعدة بيانات Chroma

صورة توضيحية:

```
from langchain_chroma import Chroma

embeddings = HuggingFaceEmbeddings(model_name="intfloat/multilingual-e5-small")

# Check if the Chroma DB exists
if os.path.exists(CHROMA_DIR):
    print("Load DB from Drive.....")
    vectordb = Chroma(
        persist_directory=CHROMA_DIR,
        embedding_function=embeddings
    )
else:
    print("Build DB for the first time....")
    vectordb = Chroma.from_documents(
        documents=chunks,
        embedding=embeddings,
        persist_directory=CHROMA_DIR # Save Automatically
    )


retriever = vectordb.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 6, "fetch_k": 20}
)
```

ملاحظة: أول تشغيل هيكون أبطأ لأنه بيبني قاعدة البيانات لأول مرة، بعد كده كل تشغيل هيكون أسرع.
شغل الخلية وانتظر الرسائل:

- "Build DB for the first time"

أو

- "Load DB from Drive"

صورة توضيحية:

```
... Build DB for the first time....
```

الخطوة رقم ٦

تجهيز نموذج ChatGPT وسلسلة الـ RAG

الكود هنا يعمل:

- تعريف دالة format_docs
- تعريف الـ LLM باستخدام "gpt-1.4-mini"
- كتابة system prompt للمساعد:
  خلي البرومبت واضح راجع البرومبت بالصورة" أنت مساعدة خدمة عملاء...."
  كل ما كان البرومت بتاعك منظم اكتر وهدفه واضح كل ما كان النموذج أذكي في استخراج المعلومات
  وأنصحك بكتابة البرومبت باللغة الإنجليزية أفضل حتي وان كان الاستخدام الأساسي للنموذج هو الرد باللغة العربية

- بناء سلسلة RAG من:
  - الـ retriever
  - الـ prompt
  - الـ LLM

شغل الخلية، لو مفيش Errors يبقى الموديل جاهز للرد على الأسئلة من قاعدة البيانات.
صورة توضيحية:

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableParallel, RunnablePassthrough

def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

llm = ChatOpenAI(model="gpt-4.1-mini", temperature=0.2)

system_prompt = """
أنت مساعد خدمة عملاء لشركة إلكترونيات.
استخدم فقط المعلومات الموجودة في النص أدناه.
لو المعلومة غير موجودة قل أنك لا تعرف
واقترح التواصل مع الدعم البشري.
طابق لغة المستخدم (عربي أو إنجليزي).
اجعل الردود قصيرة وواضحة.

{context}
"""

prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    ("human", "{question}")
])

rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
)
```

الخطوة رقم ٧

تشغيل واجهة Gradio والتجربة

الخلية الأخيرة تحتوي على:

- دالة respond أو chat_with_bot
- واجهة Gradio باستخدام gr.Blocks
- سطر تشغيل الواجهة: demo.launch(share=True)

صورة توضيحية:

```python
import gradio as gr


def respond(message, chat_history):
    """
    This function is called when 'submit' is pressed.
    """

    # Check that the message is not empty
    if not message or not message.strip():
        return chat_history, ""

    # 1. Invoke the RAG chain with only the current question
    try:
        response = rag_chain.invoke(message)
        bot_reply = response.content
    except Exception as e:
        bot_reply = f"Sorry, an error occurred: {str(e)}"

    # 2. Add the new question and answer to the chat history in 'messages' format
    if chat_history is None:
        chat_history = []

    # Add the user's message
    chat_history.append({"role": "user", "content": message})
    # Add the bot's reply
    chat_history.append({"role": "assistant", "content": bot_reply})

    return chat_history, ""

with gr.Blocks() as demo:
    gr.Markdown("## 🤖 Smart Customer Support (RAG Bot)")

    chatbot = gr.Chatbot(label="Customer Support AI Assistant")

    msg = gr.Textbox(
        placeholder="اكتب استفسارك هنا...",
        label="رسالتك"
    )

    clear = gr.Button("مسح الدردشة")

    # Link the 'respond' function to the interface
    msg.submit(respond, [msg, chatbot], [chatbot, msg])

    # Link the 'clear' button
    clear.click(lambda: ([], ""), None, [chatbot, msg])

demo.launch(share=True)
```

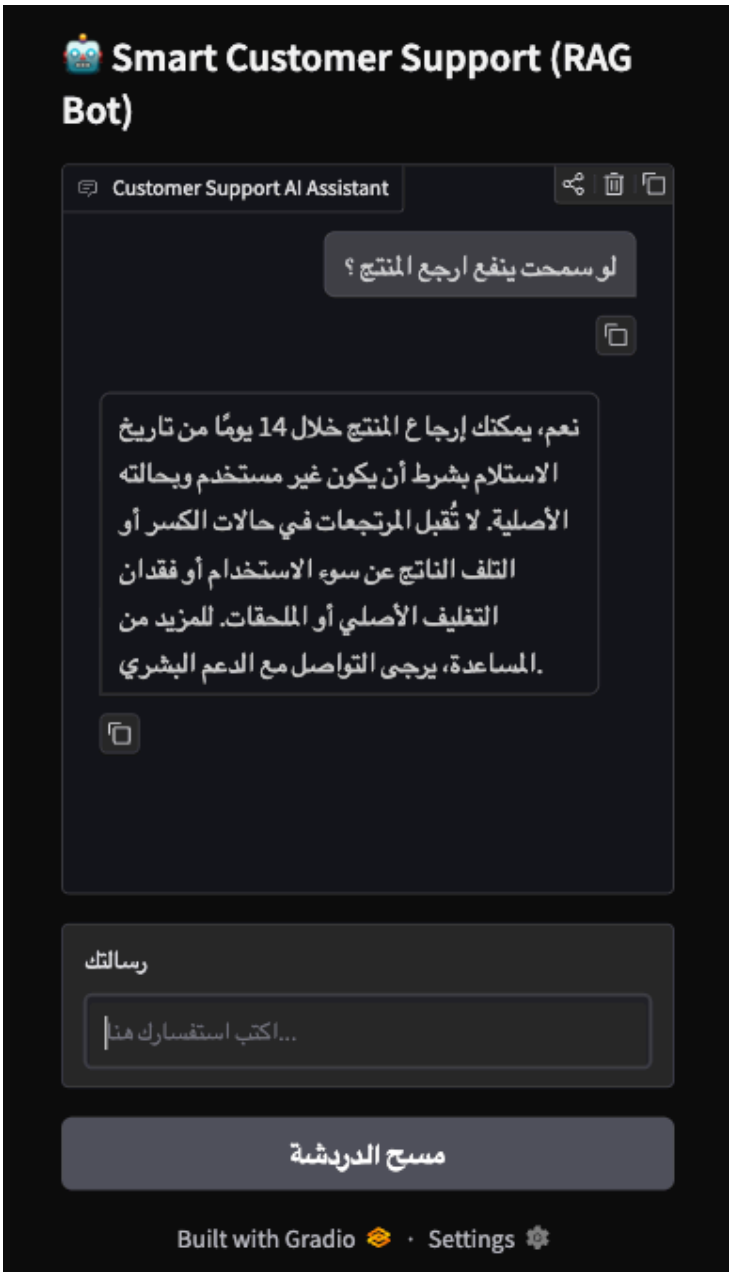
بعد التشغيل سيظهر لك رابط، افتحه وهتلاقي واجهة محادثة جاهزة.

جرب أسئلة مثل:

- ما هي سياسة الاسترجاع؟
- هل يوجد شحن مجاني؟
- مدة الضمان قد إيه؟

لو الإجابة موجودة في ملف الـ PDF هيرد من خلال قاعدة البيانات، لو مش موجودة هيقول إنه مش لاقي معلومة واضحة.

صورة توضيحية:



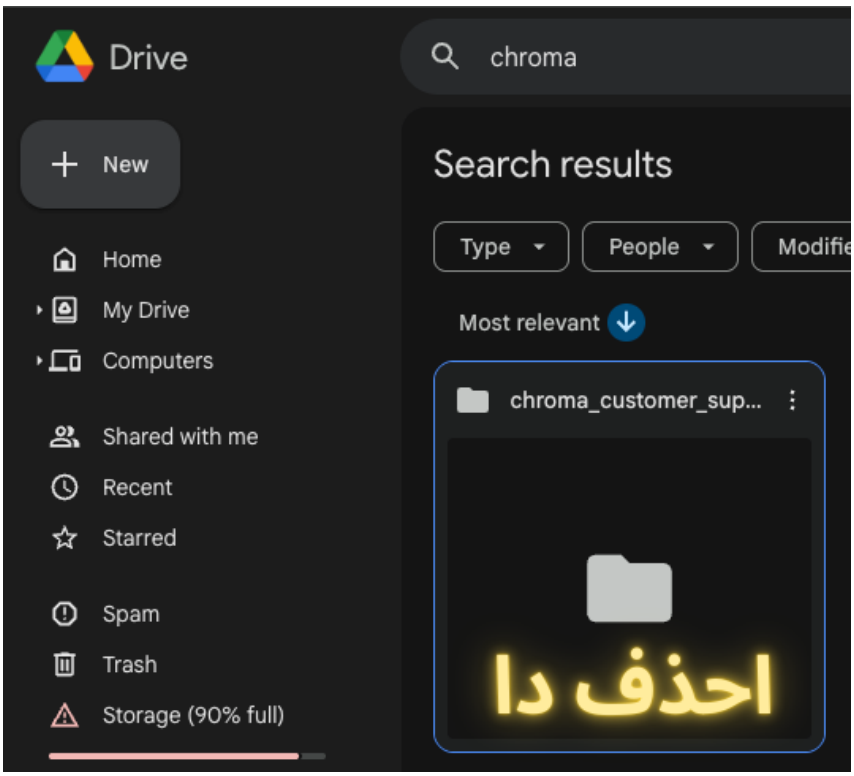
أفضل ممارسات لتحسين نتائج الـ RAG

١. استخدم ملفات PDF نصية مش صور علشان الـ Loader يقرأ النص بدون مشاكل
٢. خلّي الـ system prompt واضح قول للموديل إنه يجاوب فقط من النص المسترجع وده يقلل الهلوسة
٣. قسّم النصوص بشكل مناسب لا صغير جداً ولا كبير Chunk Size بين 600 إلى 1200 ممتاز لمعظم المشاريع
٤. احتفظ بقاعدة البيانات على Google Drive علشان متبنيهاش من أول وجديد كل مرة
٥. لما تعمل تحديث لـ PDF امسح فولدر الـ CHROMA_DIR عشان يعيد بناء قاعدة البيانات بناءً على الملف الجديد

إضافة ملف جديد لـ RAG

لو حبيت تضيف PDF جديد:

١. ارفعه على Google Drive
٢. غير قيمة PDF_PATH
٣. احذف فولدر قاعدة البيانات القديم من جوجل درايف **صورة توضيحية:**



٤. شغل الخلايا بالترتيب وهيتبني كل شيء تلقائياً

تم بحمد الله